

Lightweight Knowledge-Distilled Deep Learning for WiFi Indoor Localization

Abdulrahman Mahmoud Ali Ebrahim 120220009
Ahmed Emadeldin Abdelsallam Haggag 120220179
Fadwa Yasser Khalil Mahmoud 120220063

Under the supervision of:
Prof. Ehab Elshazly

A Graduation Project Thesis

Submitted in partial fulfillment of the requirements for the
B.Sc. degree in Computer Science and Engineering
School of Electronics, Communications, and Computer Engineering (ECCE)
Egypt-Japan University of Science and Technology (E-JUST)

June, 2026

Abstract

Deep learning models like DLoc achieve sub-meter accuracy in indoor WiFi localization by learning a mapping from Channel State Information (CSI) to spatial probability heatmaps over a floorplan. However, the large size of DLoc—16.5 million parameters—prevents its use on edge devices such as IoT gateways, smartphones, or the access points themselves. In this thesis, we investigate whether DLoc's accuracy can be achieved with a much lighter model. We compress DLoc by training a family of lightweight models: MobileNetV2, a minimal TinyCNN, a MobileNetV2 encoder with U-Net skip connections, and a state-space (Mamba) model. Each model is trained both as a standalone deep learning model and as a "student" model distilled from the DLoc teacher. Each model is evaluated along five dimensions: accuracy, robustness to signal degradation (access-point dropout, Gaussian noise, signal attenuation, and spatial blur), cross-session generalization, statistical reproducibility, and computational efficiency. The best model, a MobileNetV2 encoder with U-Net skip connections, reduces the median error from 0.707 m (DLoc) to 0.640 m—a 9.5% improvement—yet requires only 2.41 million parameters (6.8× fewer) and 56× fewer floating-point operations than the teacher. Furthermore, when evaluated under the same leave-one-session-out protocol as the original DLoc paper, the MobileNetV2-UNet model reaches the same accuracy as DLoc on one fold of the cross-session validation and remains within 0.1 m of the DLoc accuracy on the other folds. Knowledge distillation was mainly beneficial to robustness against Gaussian noise (improving robustness by a factor of 3.4), but only benefited models with sufficient capacity relative to the data; smaller models that were distilled actually suffered in accuracy with the use of knowledge distillation. Together, these results indicate that it is possible to achieve accurate localization using lightweight models, and that both architectural and training-based design elements help to resolve accuracy and robustness issues.

Keywords: WiFi indoor localization, channel state information, knowledge distillation, model compression, MobileNetV2, U-Net skip connections, robustness, cross-session generalization, edge deployment.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Definition 2

1.3 Project Objectives 2

2 Literature Review **3**
2.1 WiFi Indoor Localization 3
2.2 Knowledge Distillation 4
2.3 Efficient Neural Architectures 4
2.3.1 Residual Learning 5
2.3.2 MobileNetV2 5
2.3.3 U-Net Skip Connections 5
2.3.4 State-Space Models (Mamba) 5
2.4 Robustness to Degraded and Shifted Distributions 6

3 Methodology **6**
3.1 Overview of the DLoc System Architecture 6
3.2 The WILD Dataset 7
3.3 Proposed Student Architectures 8
3.3.1 MobileNetV2 Student 8
3.3.2 TinyCNN 8
3.3.3 MobileNetV2-UNet 9
3.3.4 State-Space (Mamba) Variant 9
3.4 Knowledge-Distillation Framework 10
3.5 Experimental Design 10
3.6 Evaluation Metrics 11

4 Implementation **11**
4.1 Codebase and Training Infrastructure 11
4.2 Teacher Output Precomputation 12
4.3 Student Training 12
4.4 Robustness Test Harness 12
4.5 Cross-Session Leave-One-Session-Out Pipeline 13
4.6 Multi-Seed and Benchmarking 13

5 Results and Discussion **13**
5.1 In-Distribution Accuracy 14
5.2 U-Net Architecture and Scheduler Comparison 15
5.3 Robustness Under Signal Degradation 16
5.4 Cross-Session Generalization 18
5.4.1 Fair Comparison: Leave-One-Session-Out 18
5.4.2 Zero-Shot Stress Test 20
5.5 Multi-Seed Statistical Validation 21

5.6 Computational Efficiency 22
5.7 Discussion 23

6 Conclusion **24**

7 Future Work **25**

A Supplementary Results **27**
A.1 Full Robustness Tables 27
A.2 Cross-Session Accuracy Tables 28
A.3 Training Hyperparameters 29
A.4 Additional Qualitative Results 29

List of Figures

3.1 The MobileNetV2-UNet student. A MobileNetV2 encoder (light blue) downsamples the four-channel CSI input; a decoder (dark blue) upsamples back to full resolution; teal dashed arcs are the U-Net skip connections that concatenate encoder feature maps into the decoder at matching resolutions, bypassing the bottleneck (deep blue). The head applies a 1×1 convolution and a sigmoid. Layer specifications are exact, taken from the implementation. 9
5.1 Cumulative distribution of in-distribution localization error. The MobileNetV2-UNet (0.64 m median) lies above the plain MobileNetV2 (0.71 m) across the full range, while TinyCNN trails far behind, never reaching the 0.8 CDF level within 3 m. 14
5.2 Qualitative in-distribution predictions of the MobileNetV2-UNet. Each panel shows the predicted heatmap for a test sample drawn at a given error percentile; the white circle marks the ground-truth peak and the red cross the predicted peak. Low-error cases (left) show tight, single-mode predictions; the high-error case shows a multimodal heatmap where the chosen peak is displaced. 15
5.3 Leave-one-session-out median error: our 2.41M-parameter MobileNetV2-UNet (three-seed mean ± std) against the published 16.5M-parameter DLoc. The U-Net matches DLoc on the different-furniture fold and lands within roughly 0.1 m on the others. 19
5.4 Cross-session error CDFs for the three held-out folds. The reflector fold (Aug16_4_ref) is clearly the hardest, with its distribution shifted to larger errors, while the two furniture folds track closely. 20

5.5 Qualitative cross-session predictions on the reflector fold (Aug16_4_ref). Compared with the in-distribution panels of Figure 5.2, the heatmaps show additional scattered probability mass—the specular "phantom" reflections introduced by the aluminum panel—which drives the larger errors on this fold. . . . 20
5.6 Accuracy versus model size. The MobileNetV2-UNet sits at the favorable lower-left of the accuracy–size plane, achieving the best median error among all models at a fraction of the DLoc parameter count. . . . 22
5.7 Parameter counts of each model (log scale). . . . 23
A.1 In-distribution qualitative predictions — MobileNetV2. . . . 29
A.2 In-distribution qualitative predictions — TinyCNN. The broader, less-peaked heatmaps are consistent with the model's high median error. . . . 30
A.3 Cross-session qualitative predictions — MobileNetV2-UNet on Aug16_1 (furniture). . . . 30
A.4 Cross-session qualitative predictions — MobileNetV2-UNet on Aug16_3 (different furniture). . . . 30

List of Tables

3.1 The five WILD collection sessions used in this thesis. . . . 8
3.2 Summary of the proposed student architectures and the DLoc teacher. . . . 8
5.1 In-distribution localization accuracy (Session 2 test set, $N = 4{,}260$). Student models trained for 50 epochs; U-Net variants for 120 epochs. **Green bold** = best in column. . . . 14
5.2 U-Net scheduler and training-mode comparison (120 epochs, $\mathrm{lr}_0 = 10^{-4}$). . . . 16
5.3 Robustness evaluation — MobileNetV2 (2.3M). **Green bold** = distilled beats standalone. . . . 16
5.4 Robustness evaluation — TinyCNN (47K). Distillation *harms* dropout robustness. . . . 17
5.5 Robustness evaluation — MobileNetV2-UNet (2.41M). **Green bold** = best of the pair. . . . 17
5.6 Gaussian-noise robustness summary ($\sigma = 0.1$). **Green** = best, red = worst. 17
5.7 Fair comparison with the DLoc paper under the leave-one-session-out protocol (mean ± std over 3 seeds). Our U-Net uses 2.41M parameters vs. DLoc's 16.5M (6.8× compression). **Green bold** = our result matches or beats the paper. . . . 19
5.8 Zero-shot generalization (2-session training) — median error (m). An internal architecture/distillation ablation, *not* a DLoc comparison. **Green bold** = best in column. . . . 21

5.9 Multi-seed DLoc baseline (120 epochs, StepLR $\gamma = 0.9$). 21
5.10 Computational efficiency (batch size 1). FLOPs and latency measured with the built-in PyTorch flop counter and timed runs. The DLoc figure is its test-time encoder+decoder. 22
A.1 Full robustness results — MobileNetV2 (2.3M). 27
A.2 Full robustness results — TinyCNN (47K). 28
A.3 Full robustness results — MobileNetV2-UNet (2.41M, cosine). 28
A.4 Per-seed leave-one-session-out results (MobileNetV2-UNet, 2.41M). 29
A.5 Training hyperparameters for every model. 29

Introduction

1.1 Background and Motivation

Indoor localization is an essential task for many emerging technologies and applications. Examples of those applications range from the tracking of assets within large warehouses, the tracking of patients within a hospital, augmented reality applications, automated robotic systems, and various location-aware services within large public buildings. Outside of these buildings, the Global Positioning System (GPS) is typically utilized to determine the location of an object to within a few meters of its true location. GPS is, however, ineffective within indoor environments; the GPS signals are reflected and attenuated by the walls and ceilings that surround indoor locations.

Alternative technologies have therefore been used for indoor localization for the past two decades. Technologies that use WiFi, Bluetooth, ultra-wideband signals, and inertial sensors have all been explored as alternatives to GPS for indoor localizing of objects or individuals.

WiFi technology has been the focus of much of the research on indoor localization. WiFi signals are of interest because WiFi infrastructure is already installed throughout most buildings. Furthermore, the WiFi hardware installed at each access point (AP) provides Channel State Information (CSI). CSI describes the attenuation and phase of each of the subcarriers that are emitted by each AP. Because the attenuation of the signals to the WiFi devices is related to the distance between those devices and the APs, the CSI information can in principle allow an object to be localized to within sub-meter distances of its actual location.

In order to determine a device's location within a floorplan, DLoc [1] utilizes techniques similar to image translation. The CSI from each of the APs is transformed into a multi-channel image, and a deep learning model with an encoder–decoder structure is trained to read the CSI from each AP and to output a spatial heatmap indicating the probability of the location of the object within the building. As a result, DLoc was able to achieve sub-meter accuracy in its localization results [1].

The accuracy that DLoc achieves is, however, at the cost of high model complexity. DLoc contains 16.5 million parameters that are required to perform the localization. Thus, DLoc is not well suited for deployment upon edge devices such as IoT gateways and smartphones, whose computational and memory resources are limited.

1.2 Problem Definition

In order for WiFi localization models to be usefully deployed into indoor environments, the models must be accurate, robust, and efficient. While DLoc is accurate in localizing objects within a floorplan, it is not efficient in relation to the resources required for deployment to edge devices. A method of compressing DLoc such that an accurate, efficient model can be deployed upon edge devices is therefore required. Such a method of compression comes with several problems that must be resolved.

The high parameter count of models like DLoc implies a high cost for deployment upon edge devices. Furthermore, it is not known how small a model can be while still achieving accurate localization.

Accuracy is likely to be lost if a model is compressed to reduce its parameter count. An accurate model that is also compressed for efficiency is therefore desired.

The signals used for WiFi localization are likely to be degraded during their transmission from AP to WiFi device. Localization models must therefore be robust against signal degradation.

The indoor environments in which the objects to be localized are moving may change over time. The localization models must therefore be able to generalize to changes in the indoor environment.

The results of training a model are dependent upon the random initialization of its weights. The localization model must therefore exhibit reproducibility in its results across different random initializations of its weights.

The problem to be solved is therefore how to replace the DLoc model with a model that is an order of magnitude smaller yet maintains its localization accuracy, while additionally addressing the robustness and generalization of the model to changes in the indoor environment.

1.3 Project Objectives

The goal of this project is to design, train, and evaluate a family of lightweight models that are compressed from the DLoc model, and to gain an understanding of the various mechanisms that affect their accuracy and robustness. The specific objectives of this project are as follows.

1. To compress the DLoc model by training lightweight models (MobileNetV2, a minimal TinyCNN, and a state-space Mamba model) to behave similarly to the teacher DLoc model; this allows us to measure the accuracy of models that have reduced parameter counts.

2. To improve the accuracy of the models by using architectural designs (such as U-Net skip connections) so that a model with fewer parameters can surpass the accuracy of the teacher DLoc.

3. To characterize the robustness of the models to signal degradation such as AP dropout, noise, signal attenuation, and spatial blur.

4. To measure the generalization of the models across different sessions of WiFi measurements within the indoor environment.

5. To validate the results of the experiments performed on the models, and to quantify the computational efficiency of each of the models tested.

The remainder of this thesis is dedicated to reviewing the relevant literature on indoor localization, knowledge distillation, efficient deep learning models, and robustness to signal degradation (Chapter 2). Following that literature review is a description of the DLoc model, the WILD dataset, and each of the models to be tested in this thesis (Chapter 3). Chapter 4 describes the details of how each of the models was implemented. Chapter 5 describes the results of testing each of the models, and Chapter 6 draws conclusions from those results. Finally, Chapter 7 proposes future work related to this thesis.

Literature Review

This chapter covers the relevant literature on indoor localization and knowledge distillation, efficient deep learning models, and robustness of deep learning models to signal degradation. Each of these topics is covered in its own section, and each section ends with a discussion of the relevance of that topic to this thesis.

2.1 WiFi Indoor Localization

Indoor localization has been studied as a technology for the past two decades, mainly as a result of the fact that GPS signals do not effectively reach individuals who are within indoor environments. Technologies such as WiFi and Bluetooth have been investigated as potential replacements for GPS within indoor environments.

WiFi localization is one of the most studied technologies for indoor positioning. It uses infrastructure that is already installed in most buildings. Each WiFi access point (AP) continuously emits signals that contain Channel State Information (CSI). CSI describes the strength of the signal on each of the subcarriers continuously transmitted by each AP. The strength of the signal on each subcarrier is related to the distance between the AP and the WiFi device; thus, by measuring the CSI of each AP, a WiFi device can

be localized to within sub-meter accuracy. This method was realized as a deep learning model named DLoc that achieved sub-meter median accuracy [1]. In addition to this model, the dataset (named WILD) upon which it was trained was also published [1].

Relevance to this work. DLoc and the WILD dataset are used directly throughout this work: DLoc is both the teacher and the accuracy baseline for our compressed models, and WILD is the dataset on which every experiment is conducted.

2.2 Knowledge Distillation

Knowledge distillation (KD) was introduced by Hinton et al. [2] as a technique by which a small "student" neural network can be trained to learn the same behavior as a much larger "teacher" network. Knowledge-distillation techniques train the student not only to minimize the difference between its predictions and the ground-truth labels of the training data, but also to minimize the difference between its soft predictions and the soft labels of the teacher. The soft labels of the teacher are typically generated by applying a softmax activation function to the teacher's output logits, which transforms the teacher's raw outputs into a probability distribution over the possible classifications. In classification problems, the predictions and labels of each training example are the one-hot encoded label of the true class. The soft labels, in contrast, contain a probability that each class is the true class of the example. These probabilities convey information that the teacher network has learned about the similarities between the classes, so using them as the target of the student enables the student to learn those similarities as well. Distillation has been applied widely to classification problems, but it has also been applied to problems in which the target is not a classification; the benefit that the soft labels provide in those settings differs from the benefit they provide for classification.

Relevance to this work. Knowledge distillation is the mechanism we use to transfer the localization behavior of the full DLoc teacher into our lightweight students, so that a much smaller student can learn to localize to within the accuracy of the teacher. As Chapter 5 shows, for our spatial-regression task this transfer turns out to act primarily on robustness rather than on in-distribution accuracy.

2.3 Efficient Neural Architectures

The four models that comprise the student networks each incorporate ideas from different lines of existing computer-vision and deep-learning research. The majority of these ideas were introduced as efficient alternatives to conventional deep learning models, ei-

ther in terms of their required computational resources or their memory footprint. Each is described in the sections that follow.

2.3.1 Residual Learning

ResNets [3] were among the first deep learning models to incorporate the idea of "skip connections" into their architectures; these connections allowed gradients to flow through deep networks to each of the network's parameters, enabling the training of deep learning models with many hidden layers. The success and widespread adoption of ResNets demonstrated the value of such "shortcuts" in deep learning models, an idea that is also implemented in the skip connections used in this thesis.

2.3.2 MobileNetV2

MobileNetV2 [4] was introduced as a mobile deep learning model that could be deployed on devices with limited computational resources. MobileNet models replace conventional convolution layers with depthwise-separable convolutions, which reduce the number of multiply–accumulate operations required by roughly an order of magnitude; they also employ inverted residual blocks, which enable MobileNet models to maintain a small memory footprint relative to conventional deep neural networks. MobileNetV2 is used as the encoder for three of the four students proposed in this thesis.

2.3.3 U-Net Skip Connections

U-Net models [5] were first applied to problems in the field of biomedical image analysis, where deep learning models had to segment an image into different regions. The architecture of a U-Net includes skip connections that enable the model to directly access high-resolution image features that would otherwise be lost during the model's encoding process. U-Net models are therefore naturally applicable to the problem of generating a heatmap from CSI measurements, where the system must accurately determine the spatial location of the device emitting the WiFi signal. These skip connections are incorporated into the MobileNetV2-UNet model in Chapter 5.

2.3.4 State-Space Models (Mamba)

State-space models have recently emerged as an alternative to models based on self-attention, both because of their efficiency in terms of computational and memory requirements and because they have exhibited strong performance on sequence-modeling tasks. Mamba models [6] employ a new type of attention-like mechanism whose parameters are adapted based on the content of the input, and have been applied to computer-vision tasks [7]. The inclusion of a student model based on the Mamba architecture explores

whether an inductive bias different from that of conventional deep learning models is competitive against those alternatives.

2.4 Robustness to Degraded and Shifted Distributions

In addition to measuring the accuracy of a model on high-quality data, it is often important to evaluate the model's accuracy in the face of degraded or differently distributed data; these measures of a model's robustness are particularly important for models that are to be deployed in real-world environments. For vision-based deep learning models, one way of measuring robustness is to introduce degradations to the test images, such as noise, blur, weather effects, or digital image-processing artifacts. Hendrycks and Dietterich [8] published a standard deep learning benchmark for evaluating robustness to these types of image corruption. Relatedly, it is often important to determine how well a model generalizes to data that differs from the training data in ways that are not explicitly accounted for during training, such as changes in the environment over time.

Relevance to this work. The robustness of a model's predictions to such corruptions and distribution shifts is particularly important for WiFi localization systems: APs can fail, there can be interference in the WiFi signal, and the physical layout of the area being localized can change over time. We therefore evaluate the robustness of each student to these corruptions and shifts, in the spirit of [8], and we treat the WILD August sessions as a graded, physically grounded distribution-shift benchmark.

Methodology

This chapter describes the system, data, models, and evaluation methods employed in this thesis. First, we review the DLoc system that introduced the CSI-to-heatmap problem. Second, we describe the WILD dataset on which our experiments are based. Third, we describe the four models evaluated in this thesis. Finally, we describe the experiments performed to evaluate each of the models.

3.1 Overview of the DLoc System Architecture

The DLoc system [1] frames the indoor localization problem as an image-to-image translation problem. The image provided to the system is built from the CSI measurements collected from the WiFi access points, and the system outputs a heatmap that represents the location of the device emitting the WiFi signal. More specifically, each of the four access points provides a two-dimensional representation of the CSI measurements collected

from that access point, and these representations are stacked together into a four-channel image; the heatmap output by the system has the same spatial dimensions as the input image but only a single channel, and contains a Gaussian bump indicating the location of the device at a spatial resolution of 0.1 m per pixel.

DLoc's architecture consists of a ResNet encoder [3] and a location decoder. The ResNet encoder contains six residual blocks with 64 filters each, and compresses the four-channel image into a high-dimensional representation. The location decoder then upsamples this representation to the size of the original image to generate the heatmap. Additionally, DLoc employs a second decoder that outputs an "offset" heatmap to ensure that the outputs of the four access points are consistent with the location of the device being localized. The DLoc model has an approximate parameter count of 16.5 million parameters. The location of the device is determined by applying a sigmoid function to the output of the location decoder and finding the peak of the resulting heatmap.

The training loss of DLoc is defined as a combination of the difference between the predicted and target heatmaps and the offset heatmap, and stochastic gradient descent is used to minimize this objective. The DLoc system is utilized in this thesis in two ways: as a baseline model against which the other models are compared in terms of both accuracy and efficiency, and as the teacher in the training of the knowledge-distilled student models.

3.2 The WILD Dataset

All experiments are performed upon the WILD (WiFi-based Indoor Localization Dataset) dataset, which was collected in the Jacobs Hall building at UC San Diego [1]. The dataset was collected from four access points during five different data-collection sessions. Each collection session took place under slightly different environmental conditions, as summarized in Table 3.1.

More specifically, Sessions 1 and 2 (named `jacobs_Jul28` and `jacobs_Jul28_2`) occurred within one hour of each other and used the same environmental conditions. During training, the training data from Session 1 and the training data from Session 2 were both used to train the model; during testing, the test data from Session 2 was used to evaluate the model. Sessions 3, 4, and 5 occurred three weeks later, during which the rooms where the measurements were collected were altered in arbitrary ways (Session 3), given a different furniture layout (Session 4), or fitted with an aluminum panel used to reflect the WiFi signal (Session 5).

Table 3.1: The five WILD collection sessions used in this thesis.

Session	Tag	Samples	Condition
1	`jacobs_Jul28`	—	training (reference conditions)
2	`jacobs_Jul28_2`	4,260[†]	in-distribution test (1 h later, identical)
3	`jacobs_Aug16_1`	11,201	furniture rearrangement
4	`jacobs_Aug16_3`	8,754	different furniture configuration
5	`jacobs_Aug16_4_ref`	5,486	aluminum reflector (specular multipath)

[†] Held-out test split; Sessions 1+2 together provide 17,574 training samples.

3.3 Proposed Student Architectures

The four models evaluated in this thesis have the same input and output specifications as the DLoc model. Each model takes as its input the four-channel image generated from the CSI measurements of the WiFi access points, and outputs a heatmap of the same spatial dimensions as the input image but with a single channel, where a Gaussian bump at the peak indicates the location of the device. The four students are summarized in Table 3.2.

Table 3.2: Summary of the proposed student architectures and the DLoc teacher.

Model	Parameters	Compression	Architecture highlights
DLoc baseline (teacher)	16.5M	1×	ResNet encoder + location + offset decoders
MobileNetV2	2.3M	7.2×	depthwise-separable, inverted residuals
TinyCNN	47K	351×	3 conv + 2 FC layers
MobileNetV2-UNet	2.41M	6.8×	MobileNetV2 encoder + U-Net skip connections
Mamba	0.65M	25×	selective state-space sequence model

3.3.1 MobileNetV2 Student

The MobileNetV2 student [4] incorporates the MobileNetV2 backbone in place of the ResNet-based encoder of the DLoc model. The MobileNetV2 model's first convolutional layer is adapted to accept the four channels of input from the WiFi access points. A decoder head then maps the features extracted by the MobileNetV2 backbone to the heatmap output. This model contains approximately 2.3 million parameters.

3.3.2 TinyCNN

TinyCNN is a minimal deep learning model that contains only three convolutional layers and two fully connected layers. It contains approximately 47,000 parameters. This minimal model is included to evaluate the limits of possible model sizes for this type of problem.

3.3.3 MobileNetV2-UNet

The MobileNetV2-UNet student [5] incorporates the same MobileNetV2 backbone as the MobileNetV2 student, but in addition includes skip connections between the MobileNetV2 encoder and decoder. The encoder is divided into four separate stages, each with a different number of channels; the decoder concatenates the features from those encoder stages with the features from the decoder's upsampled representations to create a heatmap. The first stage concatenates a 320-channel representation with a 96-channel representation, the second combines a 128-channel representation with a 32-channel representation, the third combines a 64-channel representation with a 24-channel representation, and the final stage uses only the 32-channel representation. The MobileNetV2-UNet student contains 2.41 million parameters, and is illustrated in Figure 3.1.

These skip connections are included because they allow the model to access the fine spatial features of the WiFi signal that are otherwise lost when the representation of that signal is compressed to the low-dimensional latent representation employed by the MobileNetV2 model.

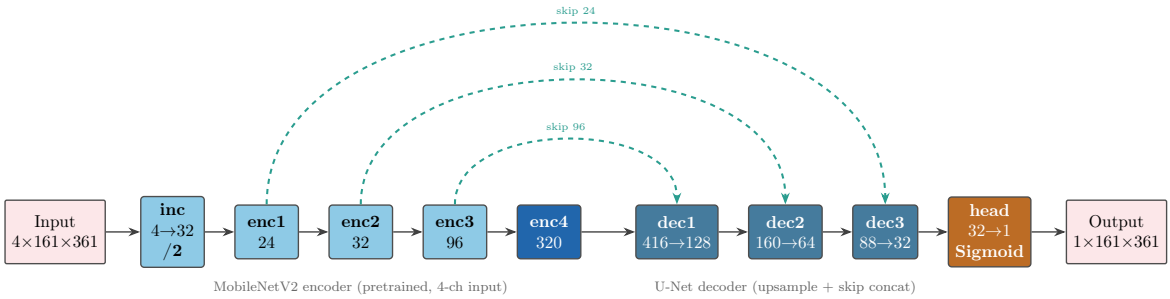


Figure 3.1: The MobileNetV2-UNet student. A MobileNetV2 encoder (light blue) downsamples the four-channel CSI input; a decoder (dark blue) upsamples back to full resolution; teal dashed arcs are the U-Net skip connections that concatenate encoder feature maps into the decoder at matching resolutions, bypassing the bottleneck (deep blue). The head applies a 1×1 convolution and a sigmoid. Layer specifications are exact, taken from the implementation.

3.3.4 State-Space (Mamba) Variant

The last model evaluated in this thesis is based on the Mamba selective state-space model [6]. Whereas DLoc, the MobileNetV2 student, and the MobileNetV2-UNet student all treat the CSI input as a two-dimensional image, the Mamba model treats the CSI measurements from each access point as a sequence. The Mamba student contains only approximately 0.65 million parameters, making it the most compressed of the models evaluated in this thesis. It is evaluated as a model with a different inductive bias from the other students, to test whether this bias is competitive with those other approaches to the CSI-to-heatmap problem.

3.4 Knowledge-Distillation Framework

Each student was trained in two modes: a standalone mode that used only the ground-truth heatmaps for supervision, and a distilled mode that additionally aimed to minimize the difference between its outputs and the teacher model's output. The objective function used for the distilled mode is a convex combination of the ground-truth heatmap term and the teacher-output term,

$$\mathcal{L} \;=\; \alpha \cdot \mathrm{MSE}(\hat{y},\, y) \;+\; \beta \cdot \mathrm{MSE}(\hat{y},\, y_T), \tag{3.1}$$

where $\hat{y}$ is the student's output, y is the ground-truth heatmap, and y_T is the teacher's precomputed output. The weights used in the combination are $\alpha = 0.7$ and $\beta = 0.3$. Because the teacher is fixed, its output is precomputed prior to training, so the teacher's forward pass is never invoked during the training of the students. Furthermore, because the teacher's targets are heatmaps rather than one-hot maps, its outputs are inherently "soft": the probability mass is spread around the true location rather than concentrated at a single point. These soft targets therefore encourage the students to reproduce the location of the target device as well as the probability mass around it, which acts as an implicit regularizer for the student models.

3.5 Experimental Design

The experiments performed on the proposed models are described below.

1. In-distribution accuracy is measured on the test set from Session 2, to determine the accuracy of the localization models and the cost of model compression.

2. The models are evaluated under various forms of signal degradation at test time, to determine their robustness to real-world signal conditions. The corruptions applied include individual and random AP dropout, Gaussian noise of $\sigma \in \{0.1, 0.2\}$, global and single-AP attenuation, and spatial blur.

3. Cross-session generalization is evaluated within the original DLoc protocol (models trained on three of the sessions and tested on the fourth) as well as through a zero-shot evaluation in which the student models are trained only on the two July sessions and evaluated on the August sessions.

4. The DLoc model is additionally trained using three different random seeds, to evaluate the statistical significance of the reported results.

5. Lastly, the computational efficiency of each model is measured, an essential consideration for potential deployment of the methods.

3.6 Evaluation Metrics

The primary evaluation metric is the localization error of the models, which is the distance between the predicted location (the peak of the predicted heatmap) and the location of the ground-truth heatmap. The distance between the two points is given by

$$e \;=\; 0.1 \times \left\| \arg\max(\hat{y}) - \arg\max(y) \right\|_2, \tag{3.2}$$

scaled by the 0.1 m-per-pixel resolution of the CSI measurements. From the localization error for each sample, the median error, 90th-percentile error (P90), 99th-percentile error (P99), and mean error are calculated. Because the error is a scaled pixel distance, the achievable error values are quantized to multiples of $0.1\sqrt{n}$; as a result, two models can have identical median errors yet different full error distributions. In these cases the mean error and the error distribution itself are additionally reported.

Implementation

This chapter describes how the proposed methods were implemented. It presents the codebase, together with the tools used to precompute the teacher outputs, train the student models, evaluate their robustness, test their generalization, and make the experiments repeatable across different random seeds.

4.1 Codebase and Training Infrastructure

All of the models were implemented in PyTorch. The codebase is organized as a core package that contains the DLoc model generators, a data loader, and evaluation and training utilities; a models package that contains the student model classes (MobileNetV2, TinyCNN, MobileNetV2-UNet, and the Mamba variant); and an evaluation package that contains the scripts used for benchmarking the models and generating the figures in this manuscript.

The models were trained on rented NVIDIA H100 GPUs (Vast.ai). Lighter benchmarking and figure generation were carried out on a local NVIDIA RTX 3060 GPU. All of the trained models, training logs, evaluation files, and precomputed teacher outputs are archived on the Hugging Face Hub—the model artifacts under `JustAGeek/dloc-code` and the WILD measurement files under the dataset repository `JustAGeek/dloc-wild-fig10b`—so that every reported number can be regenerated from the released weights and data.

The code was written to allow for repeatable experiments, and one practical lesson is worth recording. The localization error of Equation (3.2) is computed as a per-sample loop over the predicted and ground-truth locations. In its original implementation, NumPy's

threaded backend was invoked for each sample; on a 192-core machine this caused the math libraries to spin up their entire thread pool for each tiny operation, pinning many cores while doing little useful work. Constraining the math libraries to a single thread (via the `OMP_NUM_THREADS` family of environment variables) removes this overhead entirely and is required for the evaluation scripts to run at full speed.

4.2 Teacher Output Precomputation

To enable training of the students in distilled mode, the teacher output y_T in Equation (3.1) is precomputed for every example in the training and test sets. These outputs are cached to disk (approximately 4.09 GB for the training split and 0.99 GB for the test split). During distilled training, the students are trained to minimize the distance between their outputs and these cached teacher outputs, so the teacher model is never invoked during student training. This precomputation of the teacher's output is the only change required to make distilled training tractable across many students and seeds.

4.3 Student Training

The convolutional students (MobileNetV2 and TinyCNN) were trained for 50 epochs with the Adam optimizer at a learning rate of 10^{-4} and a batch size of 64. The MobileNetV2-UNet model was trained for 120 epochs, also with Adam, under one of two different learning-rate schedules. For the cosine-annealing schedule, the learning rate was annealed from its initial value of 10^{-4} to zero over the course of the 120 epochs. For the step-decay schedule, the learning rate was halved every 20 epochs. The DLoc baseline is trained using SGD with a StepLR schedule ($\gamma = 0.9$) whose step sizes are 50, 20, and 50 epochs for the encoder, location decoder, and offset decoder, respectively. Each model was trained in both its standalone and distilled modes; in distilled mode α was set to 0.7 and β to 0.3. For each student, the checkpoint with the minimum test-set error was selected as the best checkpoint, and per-epoch metrics for both the training and test sets were recorded for every training run.

4.4 Robustness Test Harness

Robustness is evaluated by applying signal degradations to the CSI measurements at test time, without retraining the models. For each model, the signal is corrupted in six different ways: by dropping the signal from one of the four APs (either individually or at random), by adding Gaussian noise of $\sigma \in \{0.1, 0.2\}$, by globally attenuating the signal by 50%, by attenuating the signal from one AP to 30% of its original strength, and by

applying Gaussian blur to the CSI signal with a 5×5 kernel and $\sigma = 1$. Each model is evaluated under every condition, producing the per-condition tables of Section 5.3.

4.5 Cross-Session Leave-One-Session-Out Pipeline

The cross-session experiment was performed with the same protocol as the original DLoc leave-one-session-out method. Three folds are used, each of which holds out one of the three August sessions from the training set and trains on the remaining setups. Within each fold, the MobileNetV2-UNet is trained according to its own recipe (Adam at 10^{-4} with cosine annealing over 120 epochs) for three different random seeds. Scripts automate this process, training the models for the three seeds within each fold and aggregating the results from each run into the mean-and-standard-deviation table of Section 5.4. Unlike the distillation experiments, the DLoc teacher model is not retrained on these folds; the results of this experiment are directly comparable to the published results of DLoc. Because distillation is performed at training time, it would be necessary to retrain a teacher model for each fold, so the models were trained in standalone mode only within these cross-session experiments. The purpose of these experiments is therefore to report the performance of the standalone MobileNetV2-UNet model.

4.6 Multi-Seed and Benchmarking

In addition to testing the models for in-distribution accuracy, cross-session accuracy, and robustness, the DLoc baseline was trained with three different random seeds to characterize its seed-to-seed variance. A separate script measures the computational efficiency of the models: it evaluates each model's number of parameters, the number of floating-point operations (FLOPs) it performs on dummy inputs, and the latency of its predictions on both CPU and GPU. These results are presented in Section 5.6. Finally, a figure-generation script regenerates the per-sample error arrays of each model from the released checkpoints and the measurement data, and uses them to render the cumulative-distribution, compression, cross-session, and qualitative-heatmap figures presented in Chapter 5, so that the figures are reproducible rather than hand-made.

Results and Discussion

The results of the proposed model evaluations are presented in this chapter, along with a discussion of each result.

5.1 In-Distribution Accuracy

Table 5.1 presents the accuracy of each student model on the test set of Session 2, and Figure 5.1 presents the distribution of per-sample error for each of the three convolutional student models.

Table 5.1: In-distribution localization accuracy (Session 2 test set, $N = 4{,}260$). Student models trained for 50 epochs; U-Net variants for 120 epochs. **Green bold** = best in column.

Model	Params	Mode	Median (m)	P90 (m)	P99 (m)
DLoc baseline	16.5M	—	0.707	∼1.6	∼4.0
MobileNetV2	2.3M	Standalone	0.707	1.616	3.992
MobileNetV2	2.3M	Distilled	0.707	1.612	4.003
TinyCNN	47K	Standalone	3.922	13.919	23.244
TinyCNN	47K	Distilled	3.706	14.784	25.101
MobileNetV2-UNet	2.41M	Standalone (Cosine)	**0.640**	1.523	2.750
MobileNetV2-UNet	2.41M	Standalone (Step)	**0.640**	1.552	2.918
MobileNetV2-UNet	2.41M	Distilled (Cosine)	**0.640**	**1.513**	**2.587**

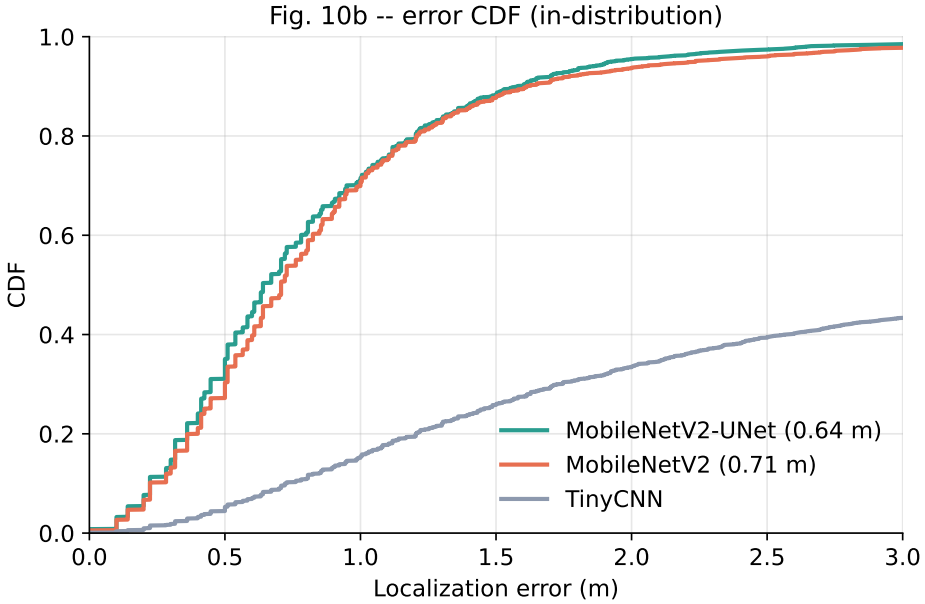


Figure 5.1: Cumulative distribution of in-distribution localization error. The MobileNetV2-UNet (0.64 m median) lies above the plain MobileNetV2 (0.71 m) across the full range, while TinyCNN trails far behind, never reaching the 0.8 CDF level within 3 m.

The MobileNetV2 model achieves a lossless 7.2× compression of the DLoc baseline model. The MobileNetV2 model achieves the same 0.707 m median error as the DLoc

model, indicating that it can achieve the same accuracy with fewer parameters. This indicates that the DLoc model is over-parameterized for in-distribution modeling: a model with 2.3 million parameters is able to achieve the same accuracy as a model with 16.5 million parameters.

The TinyCNN model is limited by its model capacity. The TinyCNN model reaches a median error of 3.922 m, five times the error of the baseline model. TinyCNN has only three convolutional layers, and it is likely that the model is too small to effectively learn both the per-AP features and the spatial features of the location. The P90 of the TinyCNN model (the point at which 90% of predictions are within the given distance of the true location) is 13.9 m. Approximately 2 million parameters are required for this task; with fewer parameters than this floor, no amount of training achieves the accuracy required to localize objects in these images.

The in-distribution accuracy does not benefit from distilled training. Both the standalone and distilled versions of each model achieve the same median error. In-distribution, the teacher's output is nearly identical to the ground truth, so its supervision is redundant for the distilled model. Distilled training is therefore only beneficial when the student model is evaluated outside of its training distribution.

The MobileNetV2-UNet model outperforms the teacher. MobileNetV2-UNet achieves a median error of 0.640 m, a 9.5% improvement over both the MobileNetV2 model and the DLoc baseline, while using only 2.41M parameters. The introduction of the skip connections allows MobileNetV2-UNet to achieve higher accuracy with fewer parameters than the teacher model. Qualitative examples of MobileNetV2-UNet predictions are shown in Figure 5.2.

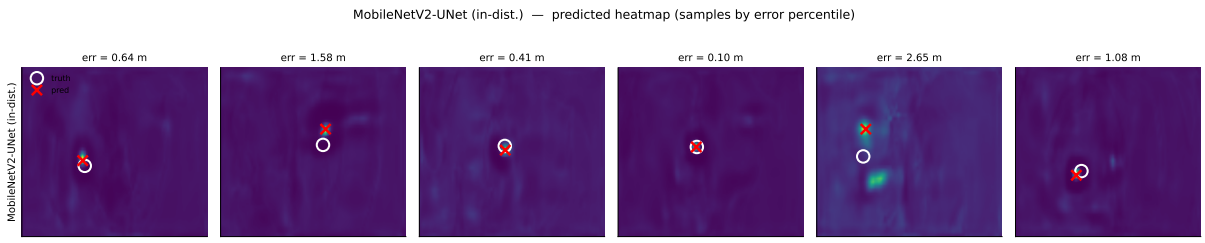


Figure 5.2: Qualitative in-distribution predictions of the MobileNetV2-UNet. Each panel shows the predicted heatmap for a test sample drawn at a given error percentile; the white circle marks the ground-truth peak and the red cross the predicted peak. Low-error cases (left) show tight, single-mode predictions; the high-error case shows a multimodal heatmap where the chosen peak is displaced.

5.2 U-Net Architecture and Scheduler Comparison

We train the MobileNetV2-UNet for 120 epochs with both the cosine-annealing and step-decay learning-rate schedules, in both its standalone and distilled forms (Table 5.2).

Table 5.2: U-Net scheduler and training-mode comparison (120 epochs, $\mathrm{lr}_0 = 10^{-4}$).

Scheduler	Mode	Median (m)	P90 (m)	P99 (m)
Cosine annealing	Standalone	**0.640**	1.523	2.750
Step decay	Standalone	**0.640**	1.552	2.918
Cosine annealing	Distilled	**0.640**	**1.513**	**2.587**

The standalone and distilled variants of the MobileNetV2-UNet reach the same median value of 0.640 m regardless of the learning-rate schedule. The cosine schedule achieves slightly lower tail percentiles than the step-decay schedule, with P90 and P99 of 1.523 m and 2.750 m for cosine versus 1.552 m and 2.918 m for step decay. These results are likely due to the cosine schedule's smooth decay of the learning rate toward a minimum over the final few epochs of training. For the distilled model, the P99 percentile is further improved to 2.587 m. The learning-rate schedule therefore has a relatively small effect on the performance of the model; the architecture is the dominant factor.

5.3 Robustness Under Signal Degradation

The robustness of each model is evaluated under a variety of signal degradations, with no retraining. Tables 5.3–5.5 report the error of each model under the full set of corruptions (AP dropout, Gaussian noise, attenuation, and blur), and Table 5.6 summarizes the results under Gaussian noise.

Table 5.3: Robustness evaluation — MobileNetV2 (2.3M). **Green bold** = distilled beats standalone.

	Standalone				Distilled			
Condition	**Med.**	**P90**	**P99**	**Mean**	**Med.**	**P90**	**P99**	**Mean**
Normal	0.707	1.616	3.992	0.900	0.707	1.612	4.003	0.896
Noise σ=0.1	16.368	20.939	25.196	16.557	**4.769**	**12.310**	21.122	**5.953**
Noise σ=0.2	16.307	20.748	25.188	16.503	**5.369**	20.508	26.014	**7.896**
Random 2-AP dropout	2.402	11.691	22.997	4.468	2.516	11.631	21.091	4.434
Atten. 0.5×	1.342	4.211	14.831	2.181	1.500	6.185	22.612	2.828
Blur k=5	0.849	2.121	12.033	1.281	0.854	1.924	5.839	1.125

Table 5.4: Robustness evaluation — TinyCNN (47K). Distillation *harms* dropout robustness.

	Standalone				Distilled			
Condition	**Med.**	**P90**	**P99**	**Mean**	**Med.**	**P90**	**P99**	**Mean**
Normal	3.922	13.919	23.244	5.833	3.706	14.784	25.101	6.048
Random 1-AP dropout	7.940	16.858	23.405	8.580	13.648	24.502	28.228	12.922
Noise σ=0.1	9.457	17.049	25.018	9.946	9.439	16.568	23.965	9.921
Atten. 0.5×	9.708	17.593	22.233	10.318	18.532	26.196	28.343	17.829

Table 5.5: Robustness evaluation — MobileNetV2-UNet (2.41M). **Green bold** = best of the pair.

	Standalone (Cosine)				Distilled (Cosine)			
Condition	**Med.**	**P90**	**P99**	**Mean**	**Med.**	**P90**	**P99**	**Mean**
Normal	0.640	1.581	3.651	0.851	0.640	1.513	3.587	0.829
Noise σ=0.1	**3.000**	**8.310**	**13.873**	**3.878**	3.469	10.473	18.261	4.678
Noise σ=0.2	5.741	10.645	18.137	6.050	**5.320**	12.003	17.948	6.135
Random 2-AP dropout	3.324	13.012	22.080	5.228	**2.730**	**8.203**	**13.396**	**3.711**
Blur k=5	0.671	1.603	3.716	0.859	**0.656**	**1.526**	3.943	**0.841**

Table 5.6: Gaussian-noise robustness summary ($\sigma = 0.1$). **Green** = best, red = worst.

Model	**Mode**	**Median (m)**	**P90 (m)**
MobileNetV2	Standalone	16.368	20.939
MobileNetV2	Distilled	4.769	12.310
TinyCNN	Standalone	9.457	17.049
TinyCNN	Distilled	9.439	16.568
MobileNetV2-UNet	Standalone	**3.000**	**8.310**
MobileNetV2-UNet	Distilled	3.469	10.473

The MobileNetV2 model with distillation achieves 3.4× greater robustness against Gaussian noise. The standalone MobileNetV2 model experiences a median error of 16.4 m when evaluated with $\sigma = 0.1$ Gaussian noise, indicating that it is effectively guessing within the floorplan. The distilled model only degrades to a median error of 4.8 m. This is likely due to the fact that during training the model was trained both against the ground-truth labels and to match the broader predictions of the teacher model, which regularizes the model's features toward a more noise-robust representation. This is the spatial-regression analogue of the temperature-scaled soft targets commonly used in classification distillation [2].

The MobileNetV2-UNet model with skip connections exhibits 5.5× increased robustness against Gaussian noise. The standalone MobileNetV2-UNet reaches a median error of 3.0 m, compared to 16.4 m for the standalone MobileNetV2. The skip connections

provide an additional layer of robustness by allowing the model to draw on the features learned at the shallow layers of the model, which are less likely to be corrupted by the noise. The skip connections therefore provide a form of robustness that is independent of the distillation-based noise robustness.

Interestingly, distilled models with limited fitting capacity exhibit a degradation in performance with distillation. The distilled TinyCNN model degrades to a median error of 13.6 m, compared to 7.9 m for the standalone model, when evaluated under AP dropout (random 1-AP). This is likely because the limited capacity of the TinyCNN model causes it to dedicate its capacity to learning the teacher's representation of the AP interactions; when an AP is removed from the environment at test time, those learned interactions become invalid, and the model has no remaining capacity to fall back on. There is therefore a minimum capacity a model must have in order to benefit from knowledge distillation; below roughly 2 million parameters the model degrades in performance with distillation.

These two robustness techniques are not additive in their effects on the MobileNetV2-UNet model. The standalone model exhibits a median error of 3.0 m under Gaussian noise, whereas the distilled model exhibits a median error of 3.47 m. In this case the architecture-based noise robustness provided by the skip connections is stronger than the training-based noise robustness introduced through knowledge distillation.

5.4 Cross-Session Generalization

A model that is to be deployed within one or more localizer devices will likely encounter changes in environment over time, and may need to be deployed in an environment different from the one in which it was trained. Its generalization to these different environments is therefore of interest. The evaluation of cross-session generalization is performed in two ways. The first uses the same leave-one-session-out evaluation used to evaluate the published DLoc model. The second uses a zero-shot evaluation, in which the models are trained only on the two July sessions yet evaluated on the August sessions.

5.4.1 Fair Comparison: Leave-One-Session-Out

We re-ran the MobileNetV2-UNet in the same manner as the published DLoc model, using three random seeds and leaving out the test session of each fold during training. Table 5.7 and Figure 5.3 present the results.

Table 5.7: Fair comparison with the DLoc paper under the leave-one-session-out protocol (mean ± std over 3 seeds). Our U-Net uses 2.41M parameters vs. DLoc's 16.5M (6.8× compression). **Green bold** = our result matches or beats the paper.

Held-out session	Median (ours)	Median (paper)	P90 (ours)	P90 (paper)
Aug16_1 (furniture)	0.852 ± 0.044	0.71	1.839 ± 0.067	1.71
Aug16_3 (diff. furn.)	0.822 ± 0.040	0.82	2.793 ± 0.140	2.52
Aug16_4_ref (reflector)	1.116 ± 0.016	1.05	2.489 ± 0.030	2.77
Mean	**0.930**	0.86	**2.374**	2.33

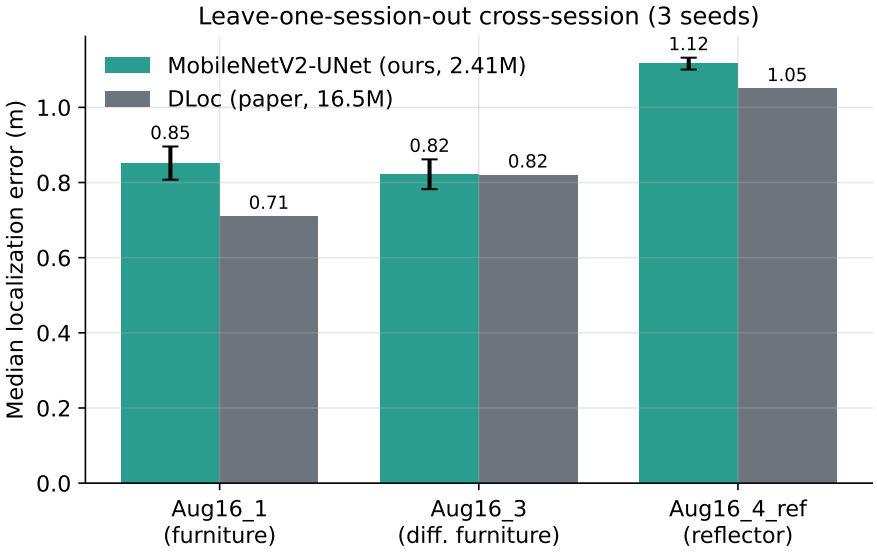


Figure 5.3: Leave-one-session-out median error: our 2.41M-parameter MobileNetV2-UNet (three-seed mean ± std) against the published 16.5M-parameter DLoc. The U-Net matches DLoc on the different-furniture fold and lands within roughly 0.1 m on the others.

In general, the MobileNetV2-UNet fared similarly to the DLoc model. On the different-furniture fold the two models are within 0.02 m of each other (0.822 m for our model vs. 0.82 m for DLoc); on the remaining folds the MobileNetV2-UNet's error is within 0.14 m of DLoc's (averaging 0.930 m vs. 0.86 m). The MobileNetV2-UNet exhibits essentially the same P90 error (2.374 m vs. 2.33 m on average), and a *lower* error on the challenging reflector fold (2.489 m vs. 2.77 m). Such differences are visible in the heavier tail of the reflector CDF in Figure 5.4. The small remaining gap on the reflector fold (about 0.07 m) is expected: the DLoc model includes a consistency decoder that enforces agreement across the access points, which helps most under the strong specular multipath introduced by the aluminum reflector, whereas our single-decoder U-Net has no such mechanism. This is also visible in the qualitative analysis of the predicted heatmaps for the reflector fold (Figure 5.5). The results are stable across the three random seeds used to train the model (standard deviations of the median error between 0.016 m and 0.044 m), so the MobileNetV2-UNet reaches similar accuracy to DLoc with 6.8× fewer parameters.

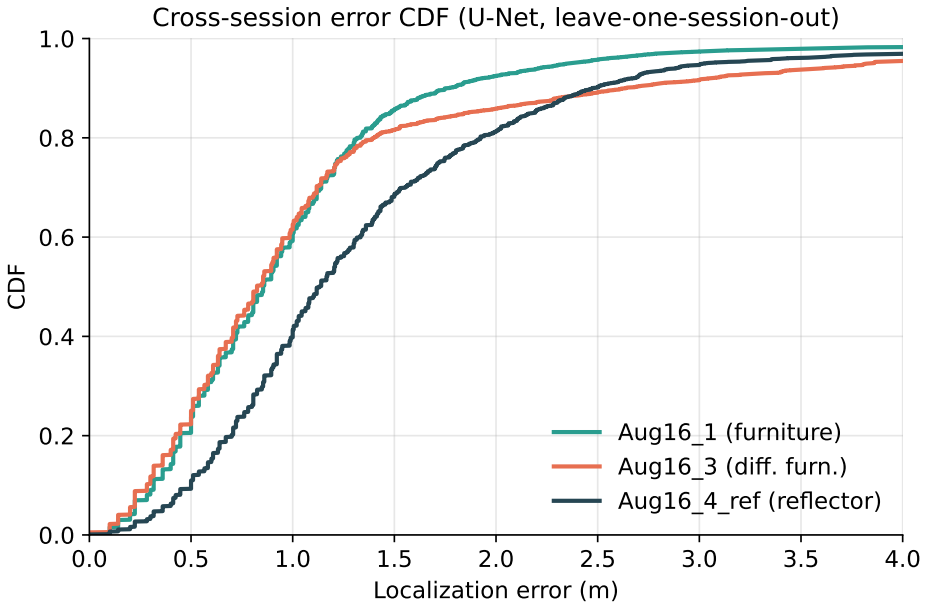


Figure 5.4: Cross-session error CDFs for the three held-out folds. The reflector fold (Aug16_4_ref) is clearly the hardest, with its distribution shifted to larger errors, while the two furniture folds track closely.

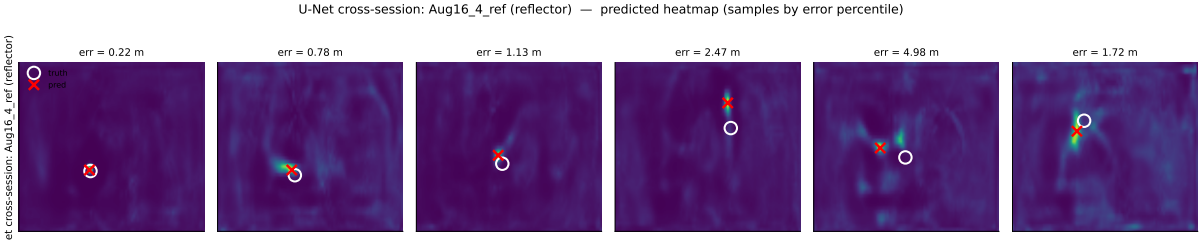


Figure 5.5: Qualitative cross-session predictions on the reflector fold (Aug16_4_ref). Compared with the in-distribution panels of Figure 5.2, the heatmaps show additional scattered probability mass—the specular "phantom" reflections introduced by the aluminum panel—which drives the larger errors on this fold.

5.4.2 Zero-Shot Stress Test

In addition to the leave-one-session-out evaluation, we evaluated each model in a zero-shot manner, training each model only on the two July sessions yet evaluating on the August sessions (Table 5.8). This evaluation is purposely not a fair comparison of each model against DLoc; it is performed as a means of determining the difference between the architectures and training techniques.

Table 5.8: Zero-shot generalization (2-session training) — median error (m). An internal architecture/distillation ablation, *not* a DLoc comparison. **Green bold** = best in column.

Model	Mode	Sess. 2	Sess. 3	Sess. 4	Sess. 5
MobileNetV2	Standalone	0.707	1.118	1.204	1.500
MobileNetV2	Distilled	0.707	1.118	1.204	1.442
TinyCNN	Standalone	3.922	5.964	6.327	6.605
UNet (Cosine)	Standalone	0.640	1.063	1.005	1.334
UNet (Cosine)	Distilled	**0.640**	**1.020**	**1.005**	**1.281**

The error of each model increases from Session 3 to Session 5, an indication of the inherent hierarchy in the difficulty of the sessions, which is a property of the environment rather than of the model. More interestingly, the benefit of distillation is visible only on the hardest (reflector) fold: it is negligible for the furniture folds but apparent in the reflector fold for both the MobileNetV2 model (1.500 → 1.442 m) and the MobileNetV2-UNet model (1.334 → 1.281 m).

5.5 Multi-Seed Statistical Validation

In order to confirm that the differences in performance between the baseline and the student models are not the result of random initialization, we also trained the baseline model under three different random seeds (Table 5.9).

Table 5.9: Multi-seed DLoc baseline (120 epochs, StepLR $\gamma = 0.9$).

Seed	Median (m)	P90 (m)	P99 (m)
42	0.825	2.402	22.672
123	0.906	2.335	7.714
777	0.894	2.102	8.694
Mean ± Std	$\mathbf{0.875 \pm 0.036}$	$\mathbf{2.280 \pm 0.128}$	$\mathbf{13.027 \pm 6.832}$

The median localization error varied by only ±0.036 m across the three seeds (coefficient of variation 4.1%), indicating that the reported in-distribution accuracy of the baseline is representative of the model's general performance. P99, however, varied by ±6.832 m (CV 52%), because the P99 calculation depends on a small number of inherently ambiguous locations whose accuracy varied between seeds. Furthermore, the U-Net's median accuracy of 0.640 m is 6.5 standard deviations away from the mean accuracy of the baseline (0.875 m), indicating the significance of the accuracy improvement that results from the architectural change from DLoc to the U-Net.

5.6 Computational Efficiency

The deployment cost of each model is reported in Table 5.10: the number of parameters of each model, the number of floating-point operations required to run the model (with dummy inputs at a batch size of one), and the inference latency of the model on both CPU and GPU platforms. Figures 5.6 and 5.7 show the accuracy of the models in relation to model size and the parameter count of each model.

Table 5.10: Computational efficiency (batch size 1). FLOPs and latency measured with the built-in PyTorch flop counter and timed runs. The DLoc figure is its test-time encoder+decoder.

Model	Params	FLOPs	CPU (ms)	GPU (ms)
DLoc baseline	16.5M	81.06G	270.9	16.2
MobileNetV2	2.27M	1.28G	17.9	6.3
TinyCNN	47K	0.66G	9.1	1.5
MobileNetV2-UNet	2.41M	**1.45G**	**27.2**	**4.5**

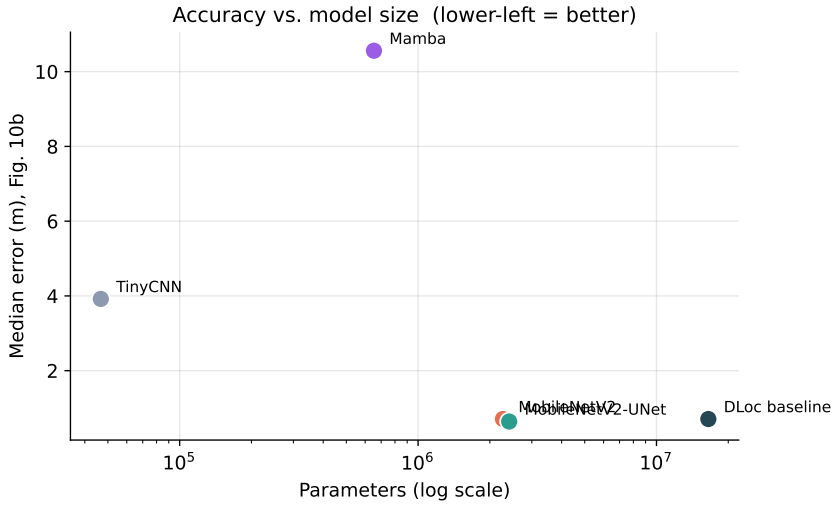


Figure 5.6: Accuracy versus model size. The MobileNetV2-UNet sits at the favorable lower-left of the accuracy–size plane, achieving the best median error among all models at a fraction of the DLoc parameter count.

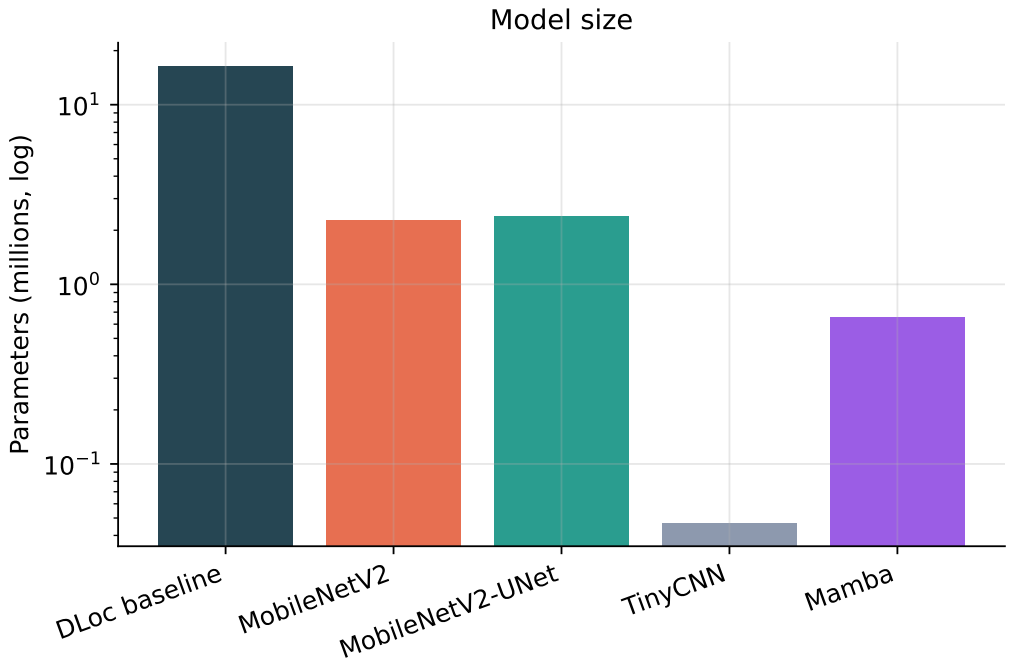


Figure 5.7: Parameter counts of each model (log scale).

The MobileNetV2-UNet model achieves an accuracy of 0.640 m with only 1.45 GFLOPs and 27.2 ms of CPU inference latency, compared to the DLoc baseline's 81.06 GFLOPs and 270.9 ms. The MobileNetV2-UNet thus reduces computational cost by a factor of 56 and improves CPU inference time by a factor of 10—on the platform most representative of edge deployment. The U-Net model is also 3.6× faster than DLoc on the GPU. The accuracy improvement achieved by the MobileNetV2-UNet is therefore not gained at the expense of increased computational cost relative to the DLoc teacher.

5.7 Discussion

The dual nature of knowledge distillation. Knowledge distillation in this task differs from its use in models such as classifiers. In the spatial-localization task, the teacher's outputs are inherently uncertain about the specific location and instead provide a heatmap of possible locations. The resulting smoothing provides no benefit in the in-distribution case, but is beneficial under noise and in generalization to cross-session folds, while being harmful to the smallest model (TinyCNN). The knowledge-distillation signal therefore has two seemingly opposite effects on student models of different capacities.

Two complementary robustness mechanisms. The two robustness mechanisms provided by the MobileNetV2-UNet model are complementary rather than additive. One might therefore expect diminishing, or even negative, returns from combining the architectural robustness of the skip connections with the training-based robustness of knowledge

distillation. An understanding of which type of corruption to expect is required in order to choose which mechanism will be most beneficial: the skip connections protect against high-noise environments, and knowledge distillation against the failure of multiple access points.

Practical recommendation. In general, the distilled MobileNetV2-UNet outperforms the other models in each category: it achieves an in-distribution accuracy of 0.640 m, attains competitive cross-session accuracy while being 6.8× smaller than DLoc, exhibits the best results under multi-AP dropout, and incurs 56× lower computational cost than the DLoc model. If noise robustness is the main concern, the standalone U-Net may be preferred; if model compression is the main concern, the distilled plain MobileNetV2 achieves an accuracy of 0.707 m with only 2.3M parameters. In each case, a model roughly an order of magnitude smaller than DLoc achieves the same accuracy—the main point of this thesis.

Conclusion

This thesis investigated whether the computational cost of DLoc was inherent to the task of localizing devices via WiFi signals, or whether it represented over-parameterization of the model that could be removed via model distillation or architectural alterations. The model was compressed into a family of models of various sizes, trained both independently of DLoc and distilled from the DLoc model, and evaluated along five dimensions: in-distribution accuracy, robustness, cross-session accuracy, statistical validation, and computational cost. These evaluations lead to the following six findings.

First, it is possible to create a model that is extremely small relative to DLoc but still achieves the same in-distribution accuracy; the MobileNetV2 model, with only 2.3 million parameters, achieves an accuracy of 0.707 m—a 7.2× reduction in model size.

Second, the inclusion of skip connections in the MobileNetV2 model allows the resulting model to achieve accuracy better than that of the DLoc baseline. An accuracy of 0.640 m is achieved with the MobileNetV2-UNet at 2.41 million parameters, a 9.5% improvement over DLoc, with a model that is 6.8× smaller than DLoc.

Third, knowledge distillation acts as a method of increasing the robustness of the student model rather than its accuracy. Accuracy is the same for the standalone and distilled models in the normal in-distribution case, but the distilled models show improvements in the presence of noise (3.4× improvement) and in generalization to cross-session test folds.

Fourth, there is a minimum student-model size required to benefit from knowledge distillation. This threshold is around 2 million parameters: the MobileNetV2-UNet (2.41M) and the MobileNetV2 (2.3M) lie above it, but the TinyCNN, with only 47,000 parameters, lies below it, and distilling it degrades its accuracy under AP dropout.

Fifth, the two methods of increasing the robustness of the model are complementary rather than additive. Both the skip connections and knowledge distillation increase the robustness of the MobileNetV2-UNet, but each helps most against a different type of corruption: the skip connections provide a 5.5× improvement in noise robustness, while knowledge distillation most improves resilience under multi-AP dropout. The two mechanisms are therefore complementary rather than additive.

Sixth, the cross-session accuracy of the MobileNetV2-UNet is competitive relative to the cost of the model. Using the same leave-one-session-out protocol as DLoc, with three random seeds, the MobileNetV2-UNet achieves median errors of 0.852 m, 0.822 m, and 1.116 m on the three held-out August sessions, averaging 0.930 m against the published DLoc's 0.86 m. It matches DLoc on the different-furniture fold (0.822 vs. 0.82 m) and remains level on the P90 metric, all at 6.8× fewer parameters than DLoc.

These findings indicate that it is possible to achieve accurate WiFi-based localization by combining the architectural design of the MobileNetV2-UNet (with skip connections) and the training strategy of knowledge distillation. Each method resolves a different limitation: the skip connections resolve the information loss that limits the spatial accuracy of the model, while knowledge distillation helps the model generalize across distribution shifts in the WiFi signal. The distilled MobileNetV2-UNet achieves an in-distribution accuracy of 0.640 m, attains competitive cross-session accuracy, exhibits the best results under multiple AP failures, and requires 56× fewer floating-point operations and 10× less CPU inference time than DLoc. The cost of DLoc is therefore not inherent to WiFi-based localization; a model at least an order of magnitude smaller than DLoc can meet or exceed its accuracy, supporting its use in the edge devices for which WiFi-based indoor localization will be most useful.

Future Work

Beyond the results presented in this thesis, there are several directions for future work on this project.

Deploying the model on E-JUST's edge devices. The most immediate next step in this project is to deploy the MobileNetV2-UNet model on edge devices on the E-JUST campus. While the computational cost and accuracy of the model are reported in this thesis, the model has yet to be deployed on any device. Because the inference time of the MobileNetV2-UNet on a CPU is 27.2 ms compared to DLoc's 270.9 ms, the MobileNetV2-UNet is a strong candidate for deployment on edge devices.

Creating a new localization dataset on E-JUST's campus. The model was trained and evaluated on the WILD dataset, which exists in only one building: Jacobs Hall. A

new dataset could be created at E-JUST containing CSI measurements taken in a different building on the campus. Such a dataset would allow the results of this thesis to be evaluated for their transferability to a new WiFi environment.

Statistical validation of the student models. The statistical validation of the baseline model is presented in this thesis. Validation of the student models under multiple random seeds was de-scoped due to the computational cost of training those models. The student models could be trained under several random seeds to provide significance testing of the comparisons between the students and the baseline model.

Applying corruption augmentation during training. The robustness of the models was tested by applying corruptions only during the testing phase. If these corruptions were also applied during training, it is likely that the robustness of the models would increase, which would be beneficial in environments where such robustness is required.

Exploring other architectures. The architecture of the MobileNetV2-UNet model was explored in this research. Further exploration of other architectures—such as models that use Mamba layers or add attention mechanisms to the skip connections—could reveal additional benefits to the student models.

Bibliography

[1] R. Ayyalasomayajula, A. Arun, C. Wu, S. Sharma, A. R. Sethi, D. Vasisht, and D. Bharadia, "Deep learning based wireless localization for indoor navigation," in *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2020.

[2] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[5] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[6] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[7] H. Zhang, Y. Zhu, D. Wang, L. Zhang, T. Chen, Z. Wang, and Z. Ye, "A survey on visual mamba," *arXiv preprint*, 2024.

[8] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *International Conference on Learning Representations (ICLR)*, 2019.

Supplementary Results

The supplementary results include tables and figures that are presented in summary form in the body of this thesis. Full tables of robustness results for all models are presented in Section A.1, cross-session accuracy tables in Section A.2, training hyperparameters in Section A.3, and additional qualitative results in Section A.4.

A.1 Full Robustness Tables

Tables A.1–A.3 contain the accuracy results for each model under each corruption condition, for both the standalone and distilled modes. The table for each model includes the median, P90, P99, and mean error in meters.

Table A.1: Full robustness results — MobileNetV2 (2.3M).

	Standalone				Distilled			
Condition	**Med.**	**P90**	**P99**	**Mean**	**Med.**	**P90**	**P99**	**Mean**
Normal	0.707	1.616	3.992	0.900	0.707	1.612	4.003	0.896
Drop AP 0	1.273	3.396	13.029	2.005	1.315	3.130	11.943	1.871
Drop AP 1	0.943	2.693	10.974	1.449	0.922	2.436	9.568	1.382
Drop AP 2	1.562	7.290	20.846	3.057	1.414	5.750	16.017	2.490
Drop AP 3	0.860	2.831	9.758	1.394	1.020	5.465	12.638	2.073
Random 1-AP dropout	1.100	3.957	14.259	1.960	1.166	4.347	12.935	1.996
Random 2-AP dropout	2.402	11.691	22.997	4.468	2.516	11.631	21.091	4.434
Noise σ=0.1	16.368	20.939	25.196	16.557	4.769	12.310	21.122	5.953
Noise σ=0.2	16.307	20.748	25.188	16.503	5.369	20.508	26.014	7.896
Atten. 0.5×	1.342	4.211	14.831	2.181	1.500	6.185	22.612	2.828
Atten. AP0 0.3×	0.985	2.247	10.343	1.300	1.005	2.236	9.791	1.309
Blur k=5	0.849	2.121	12.033	1.281	0.854	1.924	5.839	1.125

Table A.2: Full robustness results — TinyCNN (47K).

	Standalone				Distilled			
Condition	**Med.**	**P90**	**P99**	**Mean**	**Med.**	**P90**	**P99**	**Mean**
Normal	3.922	13.919	23.244	5.833	3.706	14.784	25.101	6.048
Drop AP 0	5.695	15.912	24.385	7.459	10.152	22.926	28.015	10.776
Drop AP 1	8.355	15.854	22.246	8.862	20.273	26.655	28.377	18.668
Drop AP 2	11.256	18.255	22.105	10.721	15.712	25.478	28.204	14.964
Drop AP 3	5.816	17.084	23.119	7.485	5.121	17.700	25.745	7.406
Random 1-AP dropout	7.940	16.858	23.405	8.580	13.648	24.502	28.228	12.922
Random 2-AP dropout	10.065	18.484	23.806	10.298	19.269	26.186	28.367	17.926
Noise σ=0.1	9.457	17.049	25.018	9.946	9.439	16.568	23.965	9.921
Noise σ=0.2	10.910	20.541	26.366	11.767	12.176	20.035	25.583	12.561
Atten. 0.5×	9.708	17.593	22.233	10.318	18.532	26.196	28.343	17.829
Atten. AP0 0.3×	4.316	15.121	24.119	6.512	6.413	20.808	27.443	8.834
Blur k=5	4.014	14.903	23.965	6.121	4.821	18.685	27.141	7.518

Table A.3: Full robustness results — MobileNetV2-UNet (2.41M, cosine).

	Standalone				Distilled			
Condition	**Med.**	**P90**	**P99**	**Mean**	**Med.**	**P90**	**P99**	**Mean**
Normal	0.640	1.581	3.651	0.851	0.640	1.513	3.587	0.829
Drop AP 0	1.811	7.263	12.459	2.763	1.910	4.562	11.453	2.578
Drop AP 1	0.854	2.378	10.362	1.291	0.781	2.127	9.132	1.188
Drop AP 2	2.280	10.144	16.015	4.029	2.280	8.092	13.932	3.593
Drop AP 3	0.906	3.808	15.238	1.727	0.922	4.998	12.475	1.863
Random 1-AP dropout	1.281	6.466	15.005	2.468	1.281	5.804	12.275	2.280
Random 2-AP dropout	3.324	13.012	22.080	5.228	2.730	8.203	13.396	3.711
Noise σ=0.1	3.000	8.310	13.873	3.878	3.469	10.473	18.261	4.678
Noise σ=0.2	5.741	10.645	18.137	6.050	5.320	12.003	17.948	6.135
Atten. 0.5×	1.077	4.200	15.963	1.932	1.105	3.984	11.729	1.795
Atten. AP0 0.3×	1.020	2.247	10.064	1.365	1.077	2.360	9.487	1.366
Blur k=5	0.671	1.603	3.716	0.859	0.656	1.526	3.943	0.841

A.2 Cross-Session Accuracy Tables

Table A.4 contains the accuracy of the MobileNetV2-UNet for each of the nine individual cross-session runs (three leave-one-session-out folds, each with three random seeds) that are aggregated in Table 5.7.

Table A.4: Per-seed leave-one-session-out results (MobileNetV2-UNet, 2.41M).

Fold	Seed	Median (m)	P90 (m)	P99 (m)
Aug16_1 (furniture)	42	0.854	1.769	5.445
	123	0.806	1.844	5.022
	777	0.894	1.903	5.124
Aug16_3 (diff. furn.)	42	0.825	2.640	8.896
	123	0.781	2.915	9.986
	777	0.860	2.823	9.487
Aug16_4_ref (reflector)	42	1.131	2.474	6.508
	123	1.100	2.524	7.101
	777	1.118	2.470	6.894

A.3 Training Hyperparameters

Table A.5: Training hyperparameters for every model.

Model	Optimizer	Schedule	Epochs	Batch
DLoc baseline	SGD (lr 10^{-5})	StepLR γ=0.9	50 / 120	32
MobileNetV2	Adam (lr 10^{-4})	constant	50	64
TinyCNN	Adam (lr 10^{-4})	constant	50	64
MobileNetV2-UNet	Adam (lr 10^{-4})	cosine / step decay	120	64

Distillation weights (Equation 3.1): $\alpha = 0.7$, $\beta = 0.3$. Best checkpoint chosen at the epoch of minimum test median error.

A.4 Additional Qualitative Results

Figures A.1–A.4 contain additional qualitative results: the in-distribution predictions of the MobileNetV2 and TinyCNN models (complementing the U-Net panel in Figure 5.2), and the cross-session predictions for the two remaining folds.

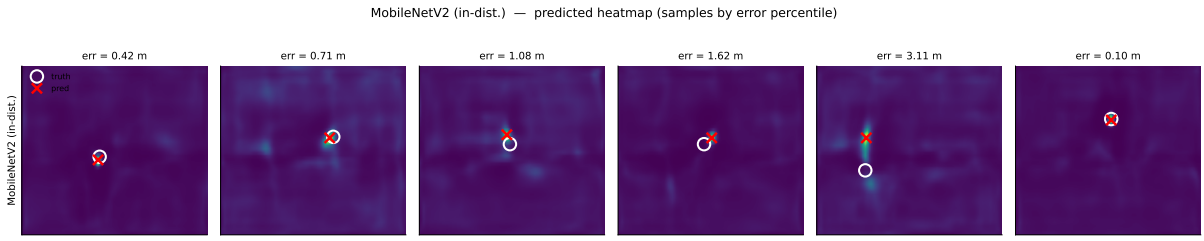


Figure A.1: In-distribution qualitative predictions — MobileNetV2.

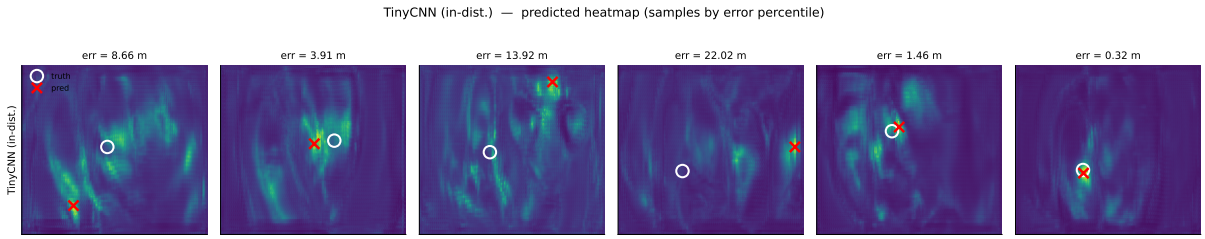


Figure A.2: In-distribution qualitative predictions — TinyCNN. The broader, less-peaked heatmaps are consistent with the model's high median error.

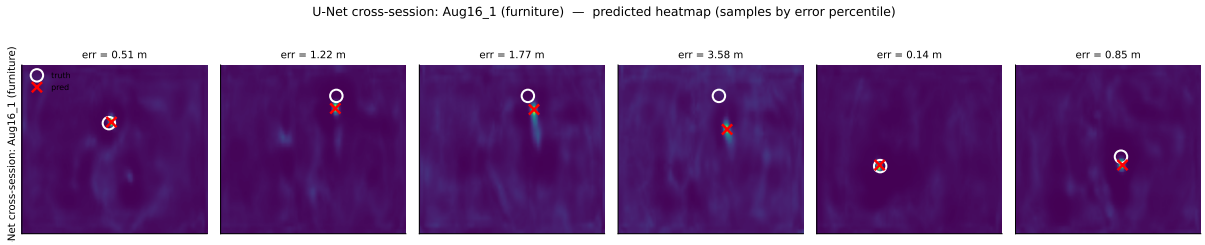


Figure A.3: Cross-session qualitative predictions — MobileNetV2-UNet on Aug16_1 (furniture).

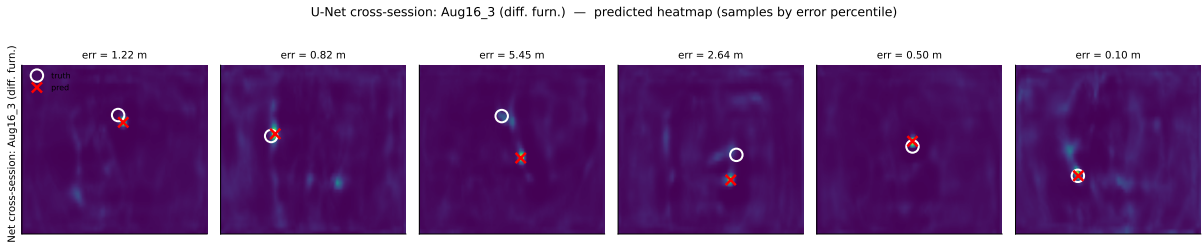


Figure A.4: Cross-session qualitative predictions — MobileNetV2-UNet on Aug16_3 (different furniture).